normative_language

Normative Language: Organizing the Normative Description of Highly Complicated Systems

Status: Draft proposal, v0.1 **Audience:** Engineers, architects, and researchers working on AI-assisted (agentic) design of complex systems — hardware, protocols, and large software.

> **Translation note:** A Chinese version of this document is maintained as `normative_language_cn.md`. The two versions must be kept consistent: any edit to one must be mirrored in the other within the same commit.

---

1. The Problem

1.1 The central question

> **How do we organize the normative description of a highly complicated system?**

By *normative description* we mean the body of statements that *govern* a design: what the system **shall** do, **must not** do, **may** do, and under what conditions. It is the authority against which an implementation is judged correct or incorrect. It is distinct from the implementation itself, from test suites (which sample it), and from tutorials or commentary (which explain it).

This question is old — standards bodies have wrestled with it for a century — but it has become acute with the arrival of agentic AI coding. An AI agent that designs or implements a system needs a normative description of the design objective. Today, no rigorous yet practical method exists for providing one.

1.2 How humans do it today

Human society encodes normative knowledge in *specification documents*: protocol specifications (IEEE 802.3, PCIe, ARM AMBA CHI), API specifications, hardware block architecture specifications, standards, and datasheets. These documents share characteristic properties:

1. **Written for human consumption.** They rely on prose, tables, timing diagrams, state-machine figures, and waveforms. Their interpretation requires human judgment and domain background.
2. **Laden with historical patches.** A mature spec like PCIe or IEEE 802.3 carries decades of errata, amendments, optional features, and deprecated clauses accumulated through revision history. The document structure reflects its editing history more than the system's logical structure.
3. **Partial by design.** Each document describes one slice of the system. A designer of an Ethernet switch must aggregate knowledge across many IEEE 802 documents (802.3 PHY/MAC, 802.1Q bridging/VLANs, 802.1AX aggregation, …) plus vendor datasheets, and the description of the *switch itself* is small relative to this mountain of referenced material. **The aggregation happens in the human reader's head and is never written down.**
4. **Cross-referenced informally.** "See Section 7.3.2" and "as defined in [802.1Q]" are resolvable by a careful human but are fragile, unverifiable, and frequently stale.

1.3 How AI-assisted design works today — and what gets lost

The current practice of human–AI collaborative design is an iterative loop:

1. The human provides an *incomplete* normative description (an initial prompt, perhaps a design doc).
2. The agent produces a partial design or a test report on the current design.
3. The human observes the output and provides *additional* prompts: clarifications, corrections, new requirements, changes of direction.
4. Repeat, possibly for hundreds of cycles, until the agent converges on a complete project.

Note what constitutes the *actual* normative description of the resulting system:

> **background reference documents + initial design document + the entire sequence of prompts and corrections**

This aggregate is the true "bible" that governed the design. And yet:

- It **does not exist in one central form**. It is scattered across chat logs, documents, and human memory.
- It **cannot be replayed canonically**. The prompt sequence is order-dependent, contains contradictions resolved implicitly by context, and is entangled with agent outputs.
- It is **usually lost**. Chat logs are discarded, not versioned, not maintained. There is no revision control, no way to ask "which requirement caused this design decision?" or "what did we decide about X in cycle 40?"
- Later prompts **silently override** earlier ones, with no record of the supersession.

The result: the system exists, but its normative description does not. Maintenance, verification, and derivative design all suffer. Every future modification restarts the archaeology.

1.4 What we need

It is imperative to define a **canonical format and method for organizing normative descriptions** that:

- can be **built up incrementally** — we cannot write it in one shot;
- is **revised throughout the project cycle**, with full **evolution history**;
- lets humans (and agents) converge on a **more complete description at the end** than at the start;
- supports **conversion of human-oriented reference documents** into the format;
- is organized **hierarchically** (a tree) for maintainability, with **easy, consistent cross-referencing** (a graph overlaid on the tree);
- is **both human- and machine-readable**;
- supports the **evolutionary, multi-cycle, iterative** agentic design / verification / physical-implementation loop with human and AI collaborators.

---

2. Design Philosophy: The Middle Path

2.1 The two failure modes

There is a spectrum of rigor for describing system behavior, and both ends fail in practice:

Failure mode A — Pure prose (the status quo). Natural-language documents written for humans: ambiguous, unmergeable, unqueryable, inconsistent, and — as argued above — not even collected in one place for AI-era workflows.

Failure mode B — Full formalization. The other extreme: describe the entire system in a formal specification language — ARM's ASL (Architecture Specification Language, "spec as code"), TLA+, or a proof assistant such as Lean or Coq. This imposes a level of rigor that is *prohibitive*:

- The effort to write and maintain the formal spec can **exceed the effort of the design itself**. (ARM can afford ASL for the ISA because the ISA amortizes across billions of chips; a project team designing one switch cannot.)
- Formal languages capture *functional* behavior well but struggle with performance intent, physical constraints, cost trade-offs, and "should be reasonable" engineering judgments — which constitute much of a real spec.
- Formalization front-loads decisions. Early in a project, the spec is *necessarily vague*; a formal language has no way to be usefully vague.
- The pool of people who can read the spec collapses.

We explicitly reject both extremes.

2.2 The middle path: structured natural language with code as a first-class citizen

Our position:

1. **Natural language remains the main vehicle.** It is the only medium expressive enough for early-stage vagueness, rationale, and non-functional intent — and now, for the first time in history, machines can read it too. LLMs change the calculus: structured natural language is no longer "machine-opaque."
2. **Structure is imposed at the *organizational* level, not the *sentence* level.** We do not constrain how a sentence is written (beyond normative-keyword conventions, §4.4);

we rigorously constrain how statements are *identified, classified, connected, and versioned*.

3. **Code is encouraged wherever it is cheaper than prose.** A 20-line Python reference model of an arbitration algorithm is more precise than a page of prose *and* faster to write. A packet-format declaration, a state table, a pseudocode fragment — these are "islands of formality" embedded in a sea of prose. We draw from the wisdom of spec-as-code without its totalizing cost.
4. **Imperfection is acknowledged and engineered for.** Human thinking is not flawless; a thoroughly complete, consistent, bug-free normative description is unattainable. The methodology's goal is not perfection but **monotonic quality improvement**: every cycle should leave the spec more complete, more consistent, and better cross-checked than before. The format must therefore support explicit markers for known gaps, open questions, and unresolved conflicts — a spec that can honestly say "this part is undecided" is more trustworthy than one that cannot.

2.3 The spec as a converging series of programs

A key idea worth elevating to a principle. Consider treating the development of a full normative spec as **writing a series of programs**:

- **P_0** is vague and high-level: mostly natural-language "comments," function signatures without bodies, `TODO`-typed holes. It "runs" only in the reader's (or agent's) head.
- **P_1, P_2, …** each provide further detail: some holes get pseudocode, some pseudocode becomes an executable reference model, some prose becomes a table becomes a data structure.
- **P_n**, the limit of the series, is a program that implements the full system — *the product itself*.

This has deep roots: it is **stepwise refinement** (Wirth, 1971) and the **refinement calculus** (Back, Morgan), where an abstract "program" containing specification statements is progressively refined into executable code, each step preserving correctness. Literate programming (Knuth) contributes the complementary insight that the prose and the code should live in one document, ordered for human understanding.

We adopt this *as a mental model and an organizing device*, with two crucial relaxations that keep it practical:

1. **Refinement steps are not formally verified.** We do not prove P_{k+1} refines P_k; we *record* the refinement relationship (clause X elaborates clause Y) and let review, testing, and agent cross-checking find violations. Rigor of bookkeeping, not rigor of proof.
2. **The series never fully collapses into the product.** In practice the spec and the implementation remain distinct artifacts at different abstraction levels; the spec's later "programs" are *reference models* and *executable acceptance checks*, not the shipping RTL or production code. The value of the series is that **every level remains present and linked** — the vague P_0 is not thrown away when P_3 exists; it remains the readable summary, and the links between levels are the traceability.

Concretely, this means the format supports **refinement layers** (§4.6): the same behavior described at multiple abstraction levels, explicitly linked, from one-sentence intent down to executable model.

---

3. Prior Art and What We Take From Each

A brief survey; each row names what we adopt and what we reject.

Source	What it is	We adopt	We reject
IETF RFCs / RFC 2119	Normative keyword conventions (MUST/SHOULD/MAY), numbered sections, errata process	Keyword discipline; the culture of "normative vs. informative" text	Monolithic flat documents; patch-by-errata
IEEE / PCIe / ARM specs	Mature multi-volume specs	The conformance-clause concept; PICS (Protocol Implementation Conformance Statements) as machine-checkable claim lists	Document structure driven by revision history; human-only diagrams
ARM ASL, TLA+, Lean	Fully formal spec languages	Executable islands; the idea that precision *can* be code	Total formalization as an entry requirement
EARS notation (Easy Approach to Requirements Syntax)	Constrained NL templates for requirements ("When ⟨trigger⟩, the ⟨system⟩ shall ⟨response⟩")	Optional sentence templates as a *lint suggestion*, never a gate	Mandatory templates for all statements
Doorstop / ReqIF / DOORS	Requirements-management: items as files in VCS, tree of documents, link validation	Item-level identity; links checked by tooling; VCS as the history mechanism	One-item-per-file granularity (too fragmented for prose-heavy specs); YAML as the authoring surface
Spec-driven development tools (GitHub Spec Kit, AWS Kiro, Tessl, 2025–26)	Markdown spec scaffolds (spec/plan/tasks) driving coding agents	Markdown as the substrate; spec-before-code workflow; the "constitution" of non-negotiables	Per-feature, per-branch specs that live only for a change request's lifetime — we need a *spec anchored over the system's lifetime*
Literate programming	Prose and code interleaved, ordered for humans	Code blocks as first-class normative content	Tangle/weave toolchain complexity
Stepwise refinement /	Program series from abstract to concrete	The layered-refinement mental model (§2.3)	Proof obligations per step

Source	What it is	We adopt	We reject
refinement calculus			
ADRs (Architecture Decision Records)	Small immutable records of decisions + context	The decision-record pattern for capturing *why*, including distilled prompt-log decisions	—
Git + Markdown ecosystem	Ubiquitous plain-text versioning	The entire persistence and history layer	Inventing a database

The gap in all prior art: nothing simultaneously (a) treats the *whole system's* normative description as a first-class, lifetime-scoped, versioned artifact, (b) stays within practical rigor, (c) is natively consumable by both humans and AI agents, and (d) provides a path for *ingesting* the existing mountain of human-oriented reference documents. That is what this proposal targets.

---

4. The Proposal: NDF — a Normative Description Format

We propose **NDF (Normative Description Format)**: a convention layered on Markdown + Git, plus a small toolchain. Nothing about it requires new file formats or servers; a plain text editor and `git` suffice to author it, which is precisely the point.

4.1 Overview of the shape

```
spec/                          # the spec root ("the bible")
  ndf.yaml                     # manifest: ID prefixes, layer names,
lint config
  00-charter/                  # what & why: scope, goals, non-goals,
glossary
  10-architecture/             # system decomposition, block diagrams
(as text + images)
  20-behavior/                 # the bulk: normative behavior clauses,
per subsystem
     20-ingress/
        pipeline.md
        parsing.md
     21-forwarding/
        ...
  30-interfaces/               # external & internal interface
contracts
  40-constraints/              # performance, resource, physical, cost
constraints
  50-verification/             # acceptance criteria, conformance
checklist (PICS-like)
```

```
  models/                       # executable reference models (code),
linked from clauses
  refs/                         # ingested external reference material
(§6)
     ieee-802.3/
     ieee-802.1q/
  decisions/                    # decision records (ADR-style), incl.
distilled prompt logs
  open/                         # open questions & known conflicts,
tracked as items
```

Three structures coexist:

1. **The tree** — the directory/heading hierarchy. This is the *ownership and maintenance* structure: every clause has exactly one home, and the tree is how humans navigate and how editing responsibility is divided.
2. **The graph** — typed cross-references between clauses (`refines`, `depends-on`, `conflicts-with`, `verifies`, `derived-from`, `couples-with`, plus plain mentions). This is the *semantic* structure; it is non-tree and freely crosses the hierarchy.
3. **The history** — Git commits, plus explicit per-clause revision markers and decision records. This is the *evolution* structure.

The discipline in one sentence: **prose lives in a tree, meaning lives in a graph, time lives in git — and stable clause IDs are the rivets holding all three together.**

4.2 The unit: the clause

The atomic normative unit is the **clause**: a heading-delimited block in a Markdown file carrying a **stable, globally unique ID**. Anatomy:

```
        ## Frame admission on ingress {#FWD-ADM-001}
        <!-- ndf: kind=req level=must layer=L1 status=stable since=0.3
-->

        When a frame arrives on an ingress port, the switch MUST admit
it to the
        forwarding pipeline only if all of the following hold:

        1. the frame passes FCS validation ([[ING-FCS-002]]);
        2. the frame length is within [minFrameSize, maxFrameSize]
           as configured per port ([[CFG-PORT-007]]);
        3. the ingress port is in `forwarding` state per the
applicable
           spanning-tree instance ([[refs/ieee-802.1q#8.6.1 | 802.1Q
§8.6.1]]).

        Frames failing any condition MUST be discarded and the per-
port
        `admissionDrops` counter ([[TEL-CNT-014]]) MUST be
incremented.

        > rationale: FCS-invalid frames must not consume pipeline
```

```
credits;
        > see [[decisions/D-0042]] for the credit-accounting
discussion.
```

Rules:

- **ID** (`{#FWD-ADM-001}`): assigned once, never reused, never renumbered, survives moves across files. IDs are per-area prefixed (declared in `ndf.yaml`) and issued by the tool (`ndf new-id FWD-ADM`), so agents and humans cannot collide.
- **Metadata comment**: machine-readable, one line, deliberately minimal:
  - `kind` — `req` (requirement), `def` (definition), `arch` (architectural statement), `constraint`, `option` (design-space parameter deferred to exploration, §4.8), `verif` (acceptance criterion), `info` (explicitly non-normative).
  - `level` — `must` / `should` / `may` / `tbd` (RFC-2119-style; `tbd` is legal and honest).
  - `layer` — refinement layer, see §4.6.
  - `status` — `draft` / `stable` / `deprecated` / `superseded-by=ID`.
  - `since` — the spec version in which the clause entered its current substance.
- **Cross-references** use `[[ID]]` (with optional `| display text`). The tool resolves them, fails the build on dangling refs, and generates a backlink index. Typed edges beyond plain reference are written as metadata: `<!-- ndf: refines=FWD-ADM-000 verifies=... -->`.
- **Normative keywords** (MUST/SHOULD/MAY/MUST NOT) are used per RFC 2119 inside `req` clauses; the linter flags `req` clauses containing none, and flags MUST/SHALL appearing in `info` clauses.

Everything outside clause blocks — introductory prose, figures, examples — is informative by default. **The normative/informative boundary is thus syntactically explicit**, which is the single biggest ambiguity in traditional specs.

4.3 Why heading-anchored clauses instead of one-item-per-file

Doorstop-style one-YAML-file-per-requirement maximizes machine tractability but destroys readability and authoring flow — nobody writes good prose one sentence per file, and prose quality is load-bearing here. Heading-anchored clauses keep files readable as documents (a human or an agent can read `parsing.md` top to bottom) while the ID discipline keeps items addressable as data. The tool can always *explode* the tree into per-item records (JSON) for querying; the authored form stays human-shaped.

4.4 Natural language, gently constrained

We do not mandate sentence templates. We do provide, via the linter, *advisory* pressure toward precision:

- EARS-style patterns offered as suggestions when a `req` clause is flagged as vague ("consider: *When ⟨trigger⟩, ⟨system⟩ shall ⟨response⟩ within ⟨bound⟩*").
- A project **glossary** (`00-charter/glossary.md`) of `def` clauses; the linter flags undefined capitalized terms of art and inconsistent synonym use ("packet" vs "frame").
- Ban-list of known ambiguity words in `must`-level clauses (*appropriately, as needed, etc., handle, support* without an object).

Advisory, not blocking: the human (or the agent, with human sign-off) can always keep the flagged sentence. Quality pressure without a rigor wall.

4.5 Code as a first-class normative citizen

A clause body may contain code blocks marked normative:

```
        ## Weighted round-robin egress scheduling {#SCH-WRR-003}
        <!-- ndf: kind=req level=must layer=L2 model=models/sch_wrr.py
-->

        Egress scheduling among queues of equal priority MUST follow
        weighted round-robin with per-queue weights `w[i]`, deficit-
counter
        variant, as specified by the reference model:

        ```python ndf:normative
        # Simplified normative model — authoritative for ordering
semantics,
        # not for performance. models/sch_wrr.py is the executable
version.
        def select_next(queues, deficit, quantum):
            for i in rotate(range(len(queues)), state.last):
                if queues[i].empty(): continue
                deficit[i] += quantum[i]
                if queues[i].head().size <= deficit[i]:
                    deficit[i] -= queues[i].head().size
                    return i
            return None
        ```
```

Conventions:

- ```python ndf:normative marks a code block as normative: it *defines* behavior, it doesn't merely illustrate. Unmarked blocks are examples (informative).
- `model=models/sch_wrr.py` links the clause to an **executable reference model** kept runnable under `models/`, with its own tests. The in-clause snippet is the readable digest; the linked model is the authoritative executable. The build runs model tests, so a broken model breaks the spec build — this is the practical substitute for formal consistency checking.
- Data-shape content (packet formats, register maps, state tables, config schemas) SHOULD be code/data rather than prose tables: a struct definition, a JSON Schema, a CSV the tool renders to a table. Machine-readable at the source, pretty at publication.

This is where "spec as code" earns its keep at near-zero marginal cost: you write code exactly where code is the cheapest precise notation, and nowhere else.

4.6 Refinement layers: the program series made concrete

Every clause carries a `layer` tag. A suggested default ladder (projects rename per `ndf.yaml`):

- **L0 — Intent.** One-to-few sentences. "The switch forwards Ethernet frames among N ports at line rate, learning addresses transparently." No mechanism.

- **L1 — Behavioral contract.** Externally observable behavior, precise but mechanism-free. Most `req` clauses live here.
- **L2 — Mechanism.** Chosen algorithms, data structures, internal decomposition. WRR scheduling, hash-based MAC tables, pipeline stages.
- **L3 — Executable model.** Reference implementations in `models/`, golden vectors, acceptance test specifications.

Rules that make this a *series* rather than a pile:

- A lower-numbered clause is refined by higher-numbered clauses via explicit `refines=` edges. `ndf trace FWD-ADM-000` prints the refinement subtree — the "program series" for that behavior.
- **All layers stay alive.** L0 is never deleted when L2 exists; it remains the summary humans and agents read first. When an L2 change contradicts its L1 parent, the linter flags the *pair* for reconciliation — sometimes the fix is at L2 (the mechanism was wrong), sometimes at L1 (the contract genuinely changed, which must be a visible, deliberate act).
- **Coverage becomes measurable.** "Which L1 contracts have no L3 acceptance criterion?" is a query (`ndf coverage`), turning "is the spec complete enough?" from a feeling into a report. Completeness is never total (§2.4 acknowledged), but its *frontier becomes visible*.
- Early in a project, the whole spec may be L0/L1 with `level=tbd` sprinkled everywhere. That is a *valid, buildable state* — vagueness is representable, which is exactly what formal methods cannot offer.

4.7 Handling imperfection explicitly

Because we accept that the spec will never be complete or fully consistent, incompleteness is *structured*, not hidden:

- **Open questions** are items in `open/`, each with an ID (`Q-017`), the clauses it blocks (`blocks=FWD-ADM-001`), and a resolution field. `ndf status` lists them. Resolving one produces a decision record and a clause edit in the same commit.
- **Known conflicts**: when two clauses are discovered to contradict, the discoverer (often an agent) adds a `conflicts-with` edge and a `Q-` item rather than silently picking a winner. The build warns but does not fail — real projects live with known conflicts for weeks, and pretending otherwise drives contradictions underground.
- **TBD holes**: `level=tbd` clauses and inline `⟨TBD: max latency bound⟩` markers are counted and reported. A release gate can require zero TBDs in `must` clauses of shipped areas — the *project* chooses its gates; the *format* just makes the holes countable.
- **Design options** (§4.8) are a distinct, richer kind of open item: a parameter whose *value or mechanism choice* is deferred to design-space exploration, but which carries a default and an allowed range, so nothing is blocked while it stays open. They are counted alongside TBDs and open questions.

4.8 Design options: parameters deferred to design-space exploration

A recurring situation in hardware and systems modeling: a quantity is genuinely undecided at model-development time, yet it is not a *gap in intent* (§4.7) — the intent is clear ("there is a reorder buffer, and it has a depth"). What is open is a **value, or a choice among mechanisms**, to be fixed later by *design-space exploration* (DSE): simulation,

sweeps, cost/performance/area trade-off studies. Examples: issue-queue and reorder-buffer depth, per-bank bandwidth, the number of concurrent threads per processing element, an arbitration policy (round-robin vs. age-based vs. priority).

The essential asymmetry: **such a quantity is a constant to the RTL implementer but a variable to the model developer.** The RTL freezes one value; the model sweeps a range and lets simulation results choose. A normative format must let both readers see the *same* clause and read off what each needs — the value to build against *and* the space still to be explored.

We give these first-class status as the `option` clause kind. An `option` clause names one design-space parameter and records:

- a **default** — the value the performance model and any early RTL should assume until DSE says otherwise (`default=`);
- an **exploration range** — the discrete set or interval the parameter is allowed to take during DSE (`explore=`);
- optional **couplings** — `couples-with=` edges to other options whose values are *not independent*, with the causal / trade-off relationship stated in prose. Parameters frequently constrain one another (e.g. a fixed total resource budget shared across threads), and exploring them one axis at a time misses the real optimum.

```
        ## Per-thread reorder-buffer depth {#PRM-ROB-001}
        <!-- ndf: kind=option level=tbd layer=L2 status=draft
since=0.4 -->
        <!-- ndf: default=64 explore=32,48,64,96,128 unit=entries -->
        <!-- ndf: couples-with=PRM-SMT-001 depends-on=PRM-IQ-001 -->

        The reorder-buffer depth allocated **per hardware thread** is
a
        design-space parameter — **TBD, to be determined by design-
space
        exploration.** It is a fixed constant in RTL and a swept
variable in
        the performance model.

        - **default:** 64 entries — assume this until DSE closes the
option.
        - **exploration range:** {32, 48, 64, 96, 128} entries.
        - **coupling:** trades off against the SMT thread count
          ([[PRM-SMT-001]]) under a fixed total ROB budget — the more
threads a
          PE supports concurrently, the smaller each thread's ROB (and
issue
          queue, [[PRM-IQ-001]]) can be. DSE MUST explore the pair
jointly,
          not each axis in isolation.

        > rationale: fixing this before the SMT width is known would
prejudge
        > the trade-off; see [[decisions/D-0101]] when the sweep
resolves it.
```

Conventions and lifecycle:

- An `option` clause is normally `level=tbd` while open. This is **not a defect** — unlike a bare `⟨TBD⟩` hole, it carries a usable default, so the model and RTL are never blocked: they proceed on the default and record that they did.
- **Inline options.** Where a parameter appears inside another clause rather than meriting its own, use the inline marker `⟨DSE: <name> default=X range=… → PRM-ID⟩`, the design-space analogue of `⟨TBD: …⟩`. Both are counted by the census (§4.7).
- **Coupling is explicit and directional in prose.** The `couples-with` edge is symmetric bookkeeping; the *nature* of the dependency — which way it pushes, under what fixed budget — is stated in the body so DSE and reviewers reason about the joint space, not one variable at a time.
- **Closing an option is a recorded event, not a silent overwrite.** When DSE picks a value, the resolution lands as a decision record plus a clause edit *in the same commit* (§5.2): `default=` becomes the chosen value, `explore=` is retained as history (or narrowed), `level` moves `tbd → must` (it is now a fixed contract), and `status` moves `draft → stable`. The RTL constant and the exploration that set it are thereafter linked.
- **Options are queryable.** `ndf options` (§8) lists every open parameter with its default, range, and coupling graph — the DSE work list, generated. `ndf coverage` counts open options alongside TBDs and open questions, so the design-space frontier is as visible as the incompleteness frontier.

This keeps the middle path (§2): the parameter's existence and intent are pinned now, in prose a human reads and a model can consume, while the one honestly-open degree of freedom — its value — is represented as a bounded, defaulted, coupled variable rather than either a false constant or an untracked hole.

---

5. Evolution and Revision Tracking

The spec is revised throughout the project; history is not an afterthought but a design driver of the format.

5.1 Git as the substrate, with spec-aware semantics on top

Plain-text Markdown in Git already gives us: full history, branching/merging for concurrent editing (crucial when multiple agents work in parallel), blame, tags as baselines, and PR review as the human-approval gate for spec changes. We add spec-aware semantics:

- **Clause-level history.** Because IDs are stable and heading-anchored, `ndf log FWD-ADM-001` reconstructs the history of *a clause* across file moves and renames — the question "how did this requirement evolve?" gets a first-class answer, which raw `git log` on files cannot give.
- **Semantic diff.** `ndf diff v0.3..v0.5` reports: clauses added / substantively modified / deprecated / superseded; edges added/removed; TBD count delta; coverage delta. This is the changelog between spec baselines, generated, not hand-written.
- **Baselines.** Tagged versions (`spec-v0.5`) are the units the *design* references: an implementation run, a verification campaign, or a tape-out records which spec

baseline it targeted. This closes the loop from artifact back to the exact governing text.
- **Supersession, not deletion.** A clause that is replaced gets `status=superseded-by=NEW-ID` and stays in the tree (optionally moved to an archive section at publication). The reasoning trail survives.

5.2 Decision records: capturing the *why*, including the prompt stream

The prompt log problem (§1.3) is solved not by archiving raw chat transcripts (unreplayable, noisy) but by **distillation into decision records**. `decisions/D-0042.md`:

```
# D-0042: Credit accounting excludes FCS-invalid frames {#D-0042}
<!-- ndf: kind=decision date=2026-07-14 affects=FWD-ADM-001,ING-FCS-002 -->

**Context.** During cycle 12 of pipeline design, agent testing showed
credit leakage when malformed frames consumed pipeline credits before
FCS check completed (test report: verif/runs/r-0231).

**Decision.** FCS validation moves ahead of credit acquisition;
FCS-invalid frames never consume credits.

**Alternatives rejected.** Post-hoc credit refund (races with
back-pressure, rejected); oversized credit pool (hides the bug).

**Source.** Human–agent session 2026-07-14; superseding instruction
in the same session overrides the cycle-9 guidance to "check FCS late."
```

The workflow rule: **any human instruction to an agent that changes design intent must terminate as either a clause edit or a decision record (usually both), in the same commit.** The agent itself drafts these — turning the session's normative content into a proposed spec commit is precisely the kind of summarization agents are good at, and the human reviews the diff. The ephemeral prompt stream is thus continuously *compiled* into the durable spec. Nothing normative lives only in a chat log.

5.3 Merge discipline

Concurrent edits by multiple humans/agents are inevitable. Mitigations, in order of importance: (1) the tree assigns each area one home file, so parallel work naturally touches disjoint files; (2) tool-issued IDs prevent identifier collisions; (3) `ndf check` runs in CI on every merge, catching dangling refs, duplicate IDs, and normative-keyword violations that textual merge cannot see; (4) semantic conflicts (two branches editing clauses connected by a `refines` edge) are surfaced by the checker as review flags.

---

6. Ingesting Human-Oriented Reference Documents

The Ethernet-switch scenario: the design's own spec is small; the referenced external specs are enormous. We need tools and methods to convert documents written for human readers into normative-description form — but converting *all* of IEEE 802.3 is neither feasible nor desirable.

6.1 Principle: import the *needed projection*, keep the pointer to the source

For each external reference, we build a **projection** under `refs/`: the subset of the external spec that this project actually depends on, restructured into NDF clauses, each carrying provenance:

```
## Frame check sequence computation {#R8023-FCS-001}
<!-- ndf: kind=req level=must origin="IEEE 802.3-2022 §3.2.9"
origin-status=verbatim -->
```

- `origin` pins the exact source clause and edition; `origin-status` is `verbatim` (faithful restatement), `paraphrase` (restructured, needs care), or `interpretation` (we resolved an ambiguity — flagged for review, and a candidate erratum to report upstream).
- The projection is *demand-driven*: an agent hitting a question about VLAN tag handling triggers ingestion of the relevant 802.1Q clauses, not the whole document. The projection grows exactly as fast as the project's real dependency frontier — this is the written-down form of the aggregation that previously happened only in the senior engineer's head (§1.2, property 3).
- Project-local clauses reference `refs/` clauses like any other (`[[R8023-FCS-001]]`), making the project's external dependency surface *enumerable*: `ndf deps refs/` lists exactly which parts of which standards the design leans on — invaluable when a standard revs.

6.2 The ingestion pipeline (agent-assisted, human-audited)

1. **Extraction.** PDF → structured text (headings, tables, figures preserved as assets). Tables → CSV/structured data where they encode state machines, formats, or parameters.
2. **Segmentation & classification.** An agent segments text into candidate clauses and classifies normative vs. informative (in mature standards, "shall"-sentences make this tractable).
3. **Restatement.** The agent restates each kept clause in NDF form, assigning `origin` and proposing `kind/level`. Diagrams get prose restatements plus the original image; state machines get table or code form.
4. **Audit.** A human (or a second, independent agent run — cheap cross-checking is a genuine advantage of the agent era) samples and reviews, prioritizing `paraphrase`/`interpretation` items. Copyright note: projections restate *technical content* for internal engineering use; teams must apply their own licensing judgment about redistribution.

This pipeline is imperfect by construction, and that is fine: a 90%-faithful, fully-linked, queryable projection of the needed 5% of a standard beats a 100%-faithful PDF that no tool and no agent can address at clause granularity.

7. The Collaborative Loop: How Humans and Agents Use This

The target workflow across design, verification, and physical implementation:

1. **Bootstrap.** Humans write `00-charter` and an L0/L1 skeleton — days, not months. `ndf init` scaffolds; ingestion (§6) starts pulling in reference projections on demand.
2. **Design cycles.** Each agent task is framed as: *baseline `spec-vX` + a work order referencing clause IDs*. The agent reads the relevant subtree plus its graph neighborhood (the clause structure is what makes retrieval precise — an agent can pull `FWD-*` and everything one `refines`/`depends-on` hop away, instead of stuffing a 400-page PDF into context).
3. **Feedback compilation.** Design outputs, test reports, and human corrections flow back as spec commits: clause edits, new L2/L3 refinements, decision records, new `Q-` items (§5.2). The spec after cycle *k* is strictly better-informed than before it.
4. **Verification.** `verif` clauses (`50-verification/`) form the conformance checklist — the PICS analogue. Each `must`-level L1 clause should eventually be `verifies`-linked from at least one acceptance criterion; `ndf coverage` reports the gap. Test failures cite clause IDs; ambiguity discovered by a failing test becomes a `Q-` item, not a shrug.
5. **Implementation cycles** (RTL, physical, software) reference clause IDs in commit messages and design reviews, so the reverse question — "which requirement drove this?" — is greppable.
6. **Publication.** `ndf publish` renders the tree to HTML/PDF with resolved cross-references, backlink indexes, trace matrices, and per-baseline changelogs — the human-friendly "book view," generated from the canonical form rather than being it.

Human and agent roles are symmetric with respect to the format — both read clauses, both propose commits — and asymmetric with respect to authority: humans approve normative changes (PR review), agents propose and cross-check. The spec is the shared memory that makes multi-agent, multi-human, multi-month collaboration coherent.

---

8. Toolchain (Minimum Viable)

Deliberately small; each piece is straightforward engineering:

Tool	Function
`ndf new-id`	Issue collision-free clause IDs
`ndf check`	Lint: dangling refs, duplicate IDs, keyword discipline, layer-consistency flags, glossary drift; runs in CI
`ndf trace ID`	Print refinement/dependency subtree for a clause
`ndf log ID`	Clause-level history across file moves
`ndf diff A..B`	Semantic changelog between baselines

Tool	Function
`ndf coverage`	L1 → verification coverage; TBD census; open-`option` census; `Q`-status
`ndf deps`	External-reference dependency surface
`ndf options`	List design-space parameters: defaults, ranges, coupling graph, open/closed status (§4.8)
`ndf export`	Explode tree to JSON records (for querying, RAG indexing, dashboards)
`ndf publish`	Render book view (HTML/PDF) with trace matrices
`ndf ingest`	Agent-assisted reference-document projection (§6.2)
`models/runner`	Execute reference models & their tests as part of the spec build

Everything operates on plain files; no server, no database. The JSON export is the bridge to whatever agent framework or search index a team uses.

---

9. Honest Limitations and Open Problems

1. **Consistency is checked socially and empirically, not proven.** Two prose clauses can contradict subtly and pass every lint. Mitigations — agent cross-reading (cheap now), executable models, verification linkage — reduce but never eliminate this. This is the accepted price of the middle path (§2.1).
2. **Distillation loses information.** Compiling prompt streams into decision records is lossy; some context dies. We judge the trade correct (durable-and-lossy beats complete-and-unusable) but teams handling disputes may want raw session archives retained as cold storage, referenced by decision records.
3. **Discipline decay.** Like all conventions, NDF degrades if commits bypass `ndf check` or agents skip the feedback-compilation step. CI enforcement and making the agent workflow *default* to spec-first are the countermeasures; culture is the real one.
4. **Granularity judgment.** How big is a clause? Too fine → bookkeeping noise; too coarse → useless addressability. We offer heuristics (one testable obligation per `req` clause) but this remains an editorial skill.
5. **Diagram-heavy content.** Timing diagrams and waveforms resist textual normalization. Interim answer: image + normative prose restatement + where possible a table/code equivalent; better answer awaits tooling (e.g., normative WaveJSON-like notations).
6. **The refinement-consistency gap.** We record `refines` edges but do not verify them; a stale L1 over a changed L2 is detectable only by review or test. A future lint (agent-powered semantic comparison of parent/child clauses) is plausible and would be a major upgrade.
7. **Ecosystem gravity.** The format's value compounds with tooling and habit; a lone adopter gets perhaps 60% of the value (structure, history, agent-retrieval precision) without the network effects.

---

10. Execution Plan

Phase 1 — Format freeze (weeks 1–2). Write `ndf.yaml` schema, clause grammar, ID rules, edge types, layer ladder as a short normative document — *itself written in NDF*, the first dogfood.

Phase 2 — Minimum toolchain (weeks 2–6). `check`, `new-id`, `export`, `trace`, `publish` in Python over a Markdown parser; CI recipe. (Doorstop and Spec Kit demonstrate every needed technique; this is assembly, not research.)

Phase 3 — Pilot on a real design (weeks 4–12, overlapping). A bounded but honest target — e.g., a 4-port L2 Ethernet switch model — with agentic design cycles run spec-first: work orders cite clause IDs, feedback compiles to spec commits, coverage tracked. The pilot's metric is §11's question list.

Phase 4 — Ingestion tooling (weeks 8–16). `ndf ingest` against the pilot's real references (802.3/802.1Q subsets); measure audit burden per ingested clause.

Phase 5 — Retrospect and revise (week 16+). Semantic diff of the spec from pilot start to end *is itself the evidence*: did the normative description converge, and would a new team (or new agent) starting from `spec-final` alone reproduce the design's intent?

11. Success Criteria

The methodology succeeds if, at the end of a pilot:

1. **The bible exists.** One versioned artifact answers "what governs this design?" — no chat-log archaeology.
2. **Provenance is queryable.** For any design decision: which clause required it; for any clause: which decision, prompt session, or external standard birthed it.
3. **Agents work from baselines.** Agent tasks cite `spec-vX` + clause IDs; two agents given the same baseline produce designs that disagree less than two agents given the ambient chat history.
4. **Evolution is legible.** `ndf diff` between any two baselines reads as a meaningful changelog a new team member can absorb.
5. **The frontier of incompleteness is visible.** TBDs, open questions, and coverage gaps are counted and trending, not lurking.
6. **The cost stayed sane.** Spec effort remained a modest fraction of total design effort — the whole point of the middle path. If maintaining the spec ever rivals doing the design, we have recreated failure mode B and must cut rigor, not add it.

Appendix A — Minimal clause grammar (informal)

```
clause        := heading id-anchor NEWLINE meta-comment NEWLINE body
id-anchor     := "{#" PREFIX "-" AREA "-" NUMBER "}"
meta-comment  := "<!-- ndf:" (key "=" value)+ "-->"
key           := "kind" | "level" | "layer" | "status" | "since"
               | "refines" | "depends-on" | "conflicts-with"
               | "verifies" | "origin" | "origin-status" | "model"
               | "affects" | "blocks" | "blocks-by" | "date" |
```

```
"author"
                | "couples-with" | "default" | "explore" | "unit"
body          := markdown, containing:
                 [[ID]] | [[ID | text]]        cross-references
                 ```lang ndf:normative ... ```  normative code islands
                 "⟨TBD: ...⟩"                  tracked holes
                 "⟨DSE: ...⟩"                  design-space parameters
(§4.8)
                 "> rationale: ..."            informative rationale
```

Grammar rules the toolchain enforces:

- **Exactly one metadata comment per clause.** All keys go into the single `<!-- ndf: ... -->` line; multiple consecutive metadata comments are an error (`ndf check`: `multiple-meta`).
- **`blocks-by`** is the inverse edge of a question's `blocks`: a clause may declare the open `Q-*` items gating it.
- **`author`** is optional in the base grammar; a project may require it on `decision`/`question` clauses via `lint: require-author: true` in `ndf.yaml`.
- **Tracked holes need the colon form.** Only `⟨TBD: ...⟩` / `⟨DSE: ...⟩` count in the census; a bare `⟨TBD⟩` token is a textual mention of a hole, not a hole.
- **Refinement layering is checked transitively.** An L2 clause may reach L1 through a chain of L2 `refines=` edges. `def`/`arch`/`constraint` clauses (and `question`/`decision` records) are exempt from layer-parenting checks. Lint findings carry three severities: **error** (violation), **warning** (judgment-dependent, promoted by `--strict`), and **note** — structural clutter such as an L1 clause that never traces to an L0 intent, informational only.

Appendix B — Worked micro-example of a refinement series

- **L0** `{#FWD-000}` — "The switch forwards frames to the port(s) where the destination is known to reside, floods when unknown, and learns source addresses." *(intent)*
- **L1** `{#FWD-LRN-001, refines=FWD-000}` — "On admitting a frame with source address S on port P, the switch MUST create or refresh the filtering-database entry (S → P) with ageing time per [[CFG-AGE-001]], except when ⟨TBD: static-entry override policy⟩." *(contract, with an honest hole)*
- **L2** `{#FWD-LRN-010, refines=FWD-LRN-001}` — "The filtering database MUST be a 4-way set-associative hash table of 16K entries keyed by {VLAN, MAC}; on set overflow the entry with the oldest refresh timestamp MUST be evicted." *(mechanism)*
- **L3** `models/fdb.py` + `{#VER-LRN-101, verifies=FWD-LRN-001}` — executable model and the acceptance criterion: "Golden vector suite `lrn-basic` MUST pass: 10K random learn/age/move events, model and DUT filtering databases equivalent at every step." *(executable)*

Every level remains in the spec; `ndf trace FWD-000` prints this series; `ndf coverage` confirms `FWD-LRN-001` is verified; the ⟨TBD⟩ appears in the census until `Q-` resolution.
```

Appendix C — Example project: a two-digit BCD chronometer

A complete, end-to-end NDF spec for a deliberately tiny design, so every mechanism from §4–§5 can be shown in full rather than in fragments. The design: a **digital chronometer** (stopwatch) with a **reset** button, a **start/stop** button, and a two-digit **seconds display driven as BCD outputs**. Small as it is, it is enough to exercise refinement layers, cross-references, normative code, decision records, open questions, and verification coverage.

C.1 Spec tree

```
spec/
  ndf.yaml
  00-charter/
     charter.md            # CHR-000, scope, non-goals
     glossary.md           # DEF-*
  20-behavior/
     counting.md           # CNT-*
     controls.md           # CTL-*
     display.md            # DSP-*
  30-interfaces/
     pins.md               # PIN-*
  40-constraints/
     timing.md             # CON-*
  50-verification/
     acceptance.md         # VER-*
  models/
     chrono.py             # executable reference model
     test_chrono.py
  decisions/
     D-0001.md             # rollover at 99, not 59
  open/
     Q-001.md              # debounce interval undecided
```

`ndf.yaml` (excerpt):

```
        project: chrono
        id-prefixes: [CHR, DEF, CNT, CTL, DSP, PIN, CON, VER, D, Q]
        layers: {L0: intent, L1: contract, L2: mechanism, L3:
executable}
```

C.2 Charter — `00-charter/charter.md`

```
        ## Chronometer intent {#CHR-000}
        <!-- ndf: kind=arch level=must layer=L0 status=stable
since=0.1 -->

        A digital chronometer measures elapsed time in whole seconds
while
```

```
        running, controlled by two momentary push-buttons — RESET and
        START/STOP — and presents the two-digit seconds count as BCD
outputs
        suitable for driving external 7-segment decoders.

        **Non-goals:** minutes/hours display, lap capture, sub-second
        resolution, power management.

        ## Definitions {#DEF-001}
        <!-- ndf: kind=def level=must layer=L1 status=stable since=0.1
-->

        - **running / stopped** — the two states of the chronometer;
the
          count advances only while *running*.
        - **BCD digit** — a 4-bit value in the range 0–9 encoding one
decimal
          digit, bit 3 = MSB.
        - **button press** — a debounced, single-clock-cycle assertion
event
          derived from a physical button ([[CTL-DEB-001]]).
```

C.3 Interfaces — `30-interfaces/pins.md`

```
        ## External pins {#PIN-001}
        <!-- ndf: kind=req level=must layer=L1 status=stable since=0.1
-->

        The design MUST expose exactly the following interface:

        ```text ndf:normative
        clk        in   1   system clock, 32.768 kHz (see CON-CLK-001)
        rst_btn    in   1   RESET button, raw, active-high,
asynchronous
        ss_btn     in   1   START/STOP button, raw, active-high,
asynchronous
        sec_lo     out  4   BCD, seconds units digit (0-9)
        sec_hi     out  4   BCD, seconds tens digit  (0-9)
        running    out  1   1 while in running state (status
indicator)
```

Button inputs are raw mechanical-switch signals; conditioning is the design's responsibility ([[CTL-DEB-001]]).

```
### C.4 Behavior — `20-behavior/`

`controls.md`:

```markdown
## Start/stop toggling {#CTL-SS-001}
```

```
<!-- ndf: kind=req level=must layer=L1 refines=CHR-000 status=stable
since=0.1 -->

Each press of START/STOP MUST toggle the state: stopped → running,
running → stopped. Stopping MUST preserve the current count; a
subsequent start MUST resume from the preserved count.

## Reset {#CTL-RST-001}
<!-- ndf: kind=req level=must layer=L1 refines=CHR-000 status=stable
since=0.1 -->

A press of RESET MUST set the count to 00 and MUST force the state to
stopped, regardless of the current state. See [[D-0001 | D-0001]] for
the rejected "reset keeps running" alternative.

## Button conditioning {#CTL-DEB-001}
<!-- ndf: kind=req level=must layer=L1 refines=CHR-000 status=draft
since=0.1 blocks-by=Q-001 -->

Each raw button input MUST be synchronized to `clk` (min. 2 flops) and
debounced such that one physical press yields exactly one press event.
A press event MUST be recognized no later than
⟨TBD: debounce interval, see Q-001⟩ after the physical press.
Simultaneous RESET and START/STOP press events MUST resolve as RESET
alone ([[CTL-RST-001]] wins).
```

counting.md:

```
        ## Counting contract {#CNT-001}
        <!-- ndf: kind=req level=must layer=L1 refines=CHR-000
status=stable since=0.2 -->

        While running, the count MUST increment by one exactly once
per
        elapsed second, with long-term rate accuracy limited only by
the
        clock source ([[CON-CLK-001]]). The count MUST hold at its
value
        while stopped. On incrementing past 99 the count MUST wrap to
00 and
        continue ([[D-0001]]).

        ## Second-tick generation {#CNT-TCK-010}
        <!-- ndf: kind=req level=must layer=L2 refines=CNT-001
status=stable since=0.2 -->

        A modulo-32768 divider on `clk` MUST generate a one-cycle
`tick`
        pulse each second. RESET ([[CTL-RST-001]]) MUST also clear the
        divider, so the first second after reset is full-length.

        ## BCD counter {#CNT-BCD-010}
```

```
        <!-- ndf: kind=req level=must layer=L2 refines=CNT-001
status=stable since=0.2 -->

        The count MUST be maintained as two cascaded decade counters
(units,
        tens), never holding a non-BCD value; on `tick` while running:
        units 9→0 carries into tens, tens 9→0 wraps the whole count to
00.
```

display.md:

```
        ## Display encoding {#DSP-001}
        <!-- ndf: kind=req level=must layer=L1 refines=CHR-000
status=stable since=0.1 -->

        `sec_hi`/`sec_lo` MUST continuously present the current count
as BCD
        ([[DEF-001]]) with no blanking, multiplexing, or intermediate
        non-BCD codes observable at the outputs; the pair MUST update
        atomically within one `clk` cycle.
```

C.5 Constraints — `40-constraints/timing.md`

```
        ## Clock source {#CON-CLK-001}
        <!-- ndf: kind=constraint level=must layer=L1 status=stable
since=0.1 -->

        The system clock is 32.768 kHz (watch crystal). The design
MUST meet
        timing at this frequency; it MAY be functional above it.
```

C.6 Executable model — `models/chrono.py` (linked from `CNT-001`)

```
        # Authoritative for control/count semantics at one-tick
granularity.
        # Debounce and clock division are below this model's
abstraction.
        class Chrono:
            def __init__(self):
                self.count, self.running = 0, False

            def press_reset(self):            # CTL-RST-001
                self.count, self.running = 0, False

            def press_startstop(self):        # CTL-SS-001
                self.running = not self.running

            def tick(self):                   # CNT-001, one call per
second
                if self.running:
                    self.count = (self.count + 1) % 100  # D-0001:
```

```
wrap at 99

        @property
        def bcd(self):                    # DSP-001
            return (self.count // 10, self.count % 10)
```

C.7 Verification — `50-verification/acceptance.md`

```
## Control sequence equivalence {#VER-CTL-001}
<!-- ndf: kind=verif level=must layer=L3 verifies=CTL-SS-
001,CTL-RST-001,CNT-001 -->

The DUT MUST match `models/chrono.py` on 1,000 randomized
sequences
of {press_reset, press_startstop, tick} (10,000 events each),
comparing `(count, running)` after every event.

## Wrap behavior {#VER-CNT-002}
<!-- ndf: kind=verif level=must layer=L3 verifies=CNT-001,CNT-
BCD-010 -->

Directed test: from reset, run 100 ticks; outputs MUST read 99
at
tick 99 and 00 at tick 100 with `running` still asserted.

## Output BCD invariant {#VER-DSP-003}
<!-- ndf: kind=verif level=must layer=L3 verifies=DSP-001 -->

Assertion, all tests: `sec_hi <= 9 && sec_lo <= 9` at every
clock
edge, including during carry propagation.
```

Note the deliberate gap: `CTL-DEB-001` has no `verifies` link yet — `ndf coverage` reports it, and it stays red until `Q-001` resolves and a debounce test is added. The frontier of incompleteness is visible, per §4.7.

C.8 Decision record and open question

`decisions/D-0001.md`:

```
# D-0001: Count wraps at 99; reset forces stop {#D-0001}
<!-- ndf: kind=decision date=2026-07-22 affects=CNT-001,CTL-
RST-001 -->

**Context.** Two-digit display; the seconds field could wrap
at 59
(clock-like) or 99 (full range). Also debated: should RESET
while
running restart the count without stopping ("flying reset")?

**Decision.** Wrap at 99 — with no minutes digit, a 59-wrap
```

```
conveys
        no extra information and wastes 40% of display range. RESET
forces
        stopped — matches user expectation of a "clear" operation.

        **Alternatives rejected.** Wrap at 59 (clock semantics without
a
        clock); flying reset (surprising, and complicates VER-CTL-
001).

        **Source.** Human–agent session 2026-07-22, cycle 2;
supersedes the
        cycle-1 prompt "make it count like a clock."

open/Q-001.md:

        # Q-001: Debounce interval {#Q-001}
        <!-- ndf: kind=question status=open blocks=CTL-DEB-001
date=2026-07-22 -->

        Datasheet for the chosen buttons not yet available; bounce
time
        unknown. Candidate: 10 ms (327 cycles @ 32.768 kHz).
Resolution will
        set the ⟨TBD⟩ in CTL-DEB-001 and add a debounce acceptance
test.
```

C.9 What the tools say

At this baseline (`spec-v0.2`), the toolchain output summarizes the state honestly (verbatim from the reference implementation, run on `examples/chrono/`):

```
$ ndf trace CHR-000
CHR-000 (L0)
├── CTL-SS-001 (L1) ── verified by VER-CTL-001
├── CTL-RST-001 (L1) ── verified by VER-CTL-001
├── CTL-DEB-001 (L1, draft, 1 TBD, blocked by Q-001)   ⚠ unverified
├── CNT-001 (L1) ── verified by VER-CNT-002, VER-CTL-001
│   ├── CNT-TCK-010 (L2)
│   └── CNT-BCD-010 (L2) ── verified by VER-CNT-002
└── DSP-001 (L1) ── verified by VER-DSP-003

$ ndf coverage
L1 must contracts:    8
  verifies-linked:    4 (4 unverified)
  unverified:         CON-CLK-001, CTL-DEB-001, DEF-001, PIN-001
TBD markers:          1
conflicts:            0
open questions:       1 of 1
```

"L1 must contracts" counts every `layer=L1 level=must` clause regardless of `kind` — definitions and constraints owe verification linkage too, or an explicit decision that they need none; the census keeps them visible either way. `CTL-DEB-001` is the one *behavioral* gap (§C.7's deliberate hole); `ndf check` additionally reports PIN-001 as structural clutter (an L1 root that never traces to L0 intent) — a note, not a violation.

Even at this toy scale the payoff pattern is visible: the "make it count like a clock" prompt from cycle 1 did not vanish into a chat log — it was superseded by a recorded decision (`D-0001`) that two clauses cite; the one genuinely undecided engineering parameter is a tracked hole, not an ambush; and an agent asked to "implement the counter" can be handed exactly `CNT-*` plus its one-hop neighborhood (`CTL-RST-001`, `CON-CLK-001`, `D-0001`) instead of a transcript.